QCalc design overview (English)

Nov 12, 2026 · @津嘉山 弘

This is an English summary of the design document set of **QCalc 1.2.0** (versionCode 3, package name `io.github.hiroshitsu.qcalc`). The seven full documents are written in Japanese. References such as “detailed design 3.6” point to the section of those documents where a topic is described in detail; section 10 lists the documents.

1. What QCalc is

QCalc is a multi-function calculator for Android. It runs **libqalculate 5.13.2**, the calculation engine of the open-source desktop calculator Qalculate!, without modifying it, and adds a user interface for phones. The user interface is in Japanese.

- **Why**: Qalculate! has versions for PCs (GTK, Qt and the command-line `qalc`) but no official Android version. The goal is to make all of its calculation features usable on an arm64 Android phone and to publish the app on Google Play. The reference device is a SHARP AQUOS sense8 (Android 16, 1080 × 2432).
- **What users do**: type expressions such as `60 mph to km/h`, `100 USD to JPY`, `solve(x^2 − 5x + 6 = 0)` or `integrate(x^2 sin(x), 0, pi)`. The syntax and the results are the same as those of `qalc`.
- **What QCalc adds to libqalculate**: only ten traditional Japanese units (坪, 反, 寸, 尺, 間, 里, 合, 升, 匁, 貫) and the matrix trace `trace()`, both added to the running engine at start-up.
- **What 1.2.0 changed**: the precision limit was raised from 1,000 to 200,000 digits, the most that works well on Android (the clipboard takes about 1 MB at a time). Long results are shown, copied and kept in full: the history keeps the newest results whole up to 1,000,000 characters in total. ans is given back with all its digits after a restart (1.1.0 rounded it to about 38 digits), and long numbers in it are read directly with GMP. A result too long to copy gives a message instead of a crash.
- **What 1.1.0 changed**: four keypads switched with tabs (123, functions, calculus and matrices), matrix results shown as tables, built-in examples (12 topics, 69 expressions), undo for a cleared expression or history, three text sizes, Japanese names and search for functions, units and constants, and a workaround for a libqalculate 5.13.2 problem that hid later warnings (section 5).

In scope	Out of scope
Everything libqalculate calculates: arithmetic and functions, units, currencies, equations (`solve`), derivatives and integrals, matrices, number bases, dates, uncertainty propagation, complex numbers, data sets, up to 200,000 digits	Devices other than arm64 (32-bit ARM, x86)
Input with the on-screen keys and the phone keyboard, live preview, history, `ans`, cancelling a calculation	gnuplot plotting and the file and command functions such as `command()`, `load()` and `export()` (QCalc draws its own graphs)
Lists of functions, variables and units with search, 2D and 3D graphs, user-defined variables and functions, exchange-rate updates, settings, examples	RPN mode, user interfaces in other languages, sync between devices
Distribution on Google Play, license texts in the app, published source code	Eigenvalues as a single function, differential equations above first order, values of infinite sums (libqalculate cannot do these)

2. Requirements in brief

Requirements definition 1.5 lists 29 functional requirements (F-01 to F-29; F-22 to F-28 were added in 1.1.0, and F-29, long results, in 1.2.0). All of them are implemented in 1.2.0.

- **Calculation** (F-01): the full Qalculate! feature set with the same syntax. libqalculate 5.13.2 is used unmodified, and the typed expression is passed on as it is.
- **Input** (F-02, F-03, F-14, F-22 to F-24): four 5 × 5 keypads plus eight keys that are always shown, the phone keyboard, and automatic conversion of full-width characters. Calculus keys insert templates such as `integrate(, 0, 1)` with the cursor where the expression goes, and an a=b key inserts the “=” of equations.
- **Results** (F-04 to F-08, F-15): a live preview 160 ms after the last key (cut off after 1.5 s), confirming with “=” (limited to 20 s), a history of up to 300 items, `ans`, reuse and copy of results, and cancelling. Inserting a result uses the exact value returned by the engine.
- **Lists and definitions** (F-09, F-11): lists of functions, variables and units from the engine, which can also be searched by Japanese names, and user-defined variables and functions saved in the engine's definition files.
- **Graphs** (F-10, F-19): 2D graphs of up to 6 functions, and 3D graphs of up to 4 surfaces on a 25 × 25, 41 × 41 or 61 × 61 grid, computed by the engine and drawn by the page.

- **Data** (F-12, F-18): exchange rates from the same three sources as Qalculate!, updated by hand or automatically after 7 days. The history, the settings, the draft and `ans` are written to files at once.
- **Usability** (F-13, F-16, F-17, F-25 to F-27): 17 settings, dark mode, a landscape layout, built-in examples to tap and try, undo, and three text sizes (1, 1.18 and 1.38 times).
- **Distribution** (F-20, F-21): a privacy policy on GitHub Pages, the full license texts of the app and its seven bundled components, and an App Bundle with Play App Signing.
- **Additions** (F-28): the Japanese units and `trace()`, added by the bridge program at start-up and never written to the user's definition files.

The non-functional requirements (requirements definition 1.6) come down to one idea: whatever is typed, the app must never freeze, and the value shown, the value saved as `ans` and the value inserted from the history must always be the same.

- **Performance**: the preview is cut off after 1.5 s, a confirmed calculation after 20 s, a 2D graph after 12 s and a 3D graph after 15 s. Every request carries its own time limit, and the engine starts together with the page.
- **Reliability**: the app never freezes and recovers from engine crashes, thanks to three layers of time limits (section 6). A request that crashed the engine is never sent again; a request is resent only when it could not be written.
- **Accuracy**: each expression is evaluated once, the results match `qalc` (309 expressions compared), and warnings are never lost, because message stops are closed before every command.
- **Data safety**: files are written to a temporary file, synced with fsync and then renamed. Unreadable definition files are moved aside, never overwritten.
- **Security and privacy**: the WebView shows only the bundled page, `command()` is disabled at build time, and the only network traffic is downloading exchange rates. No personal data is collected.
- **Compatibility**: Android 7.0 to 16 with targetSdk 36 (edge-to-edge display on Android 15+, back handling on Android 13+), and a static arm64 executable aligned for 16 KB pages.
- **Maintainability and license**: libqalculate is unmodified, all builds are reproducible with four shell scripts, the app is GPL v2 or later, and the sources of every bundled library are published.

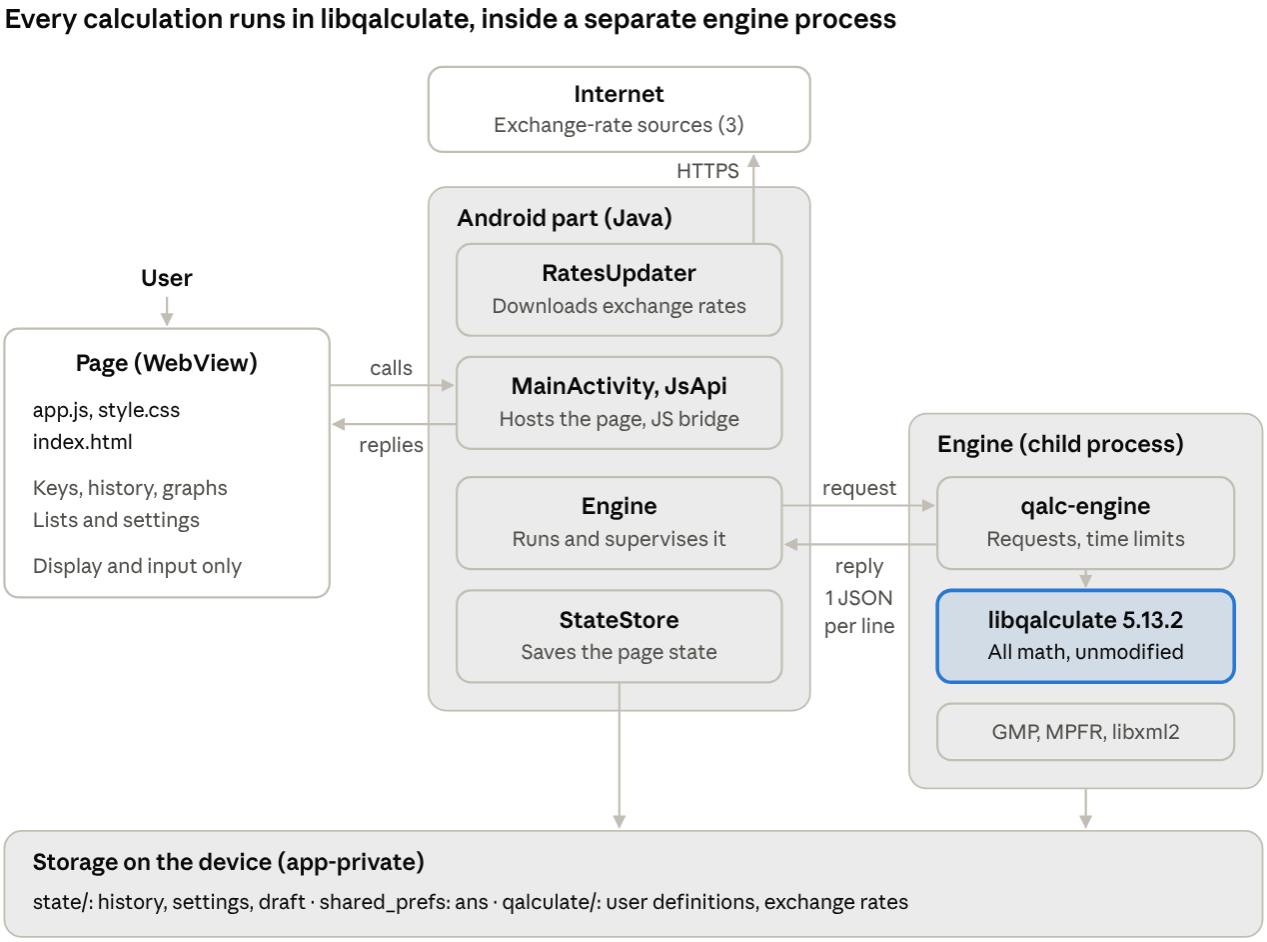
Main constraints (requirements definition 1.7):

1. The source of libqalculate is not changed. Everything specific to QCalc lives in a small bridge program that calls libqalculate's public API.

2. The engine is built with zig as a cross compiler, without the Android NDK. The APK and the App Bundle are assembled with Ubuntu's Android tools (aapt2, dx, zipalign, apksigner, jarsigner) and android.jar from the Android SDK.
3. The engine is a Linux executable stored as `lib/arm64-v8a/libqalcengine.so`. Android 10 and later do not let an app run files from its data directory, so the engine is started from the native library directory, where Android extracts it at install time.
4. targetSdk is 36, as Google Play requires for new apps, so the app supports edge-to-edge display (Android 15+) and the new back handling (Android 16).
5. Google Play signs the app (Play App Signing). The developer keeps only an upload key, which is never published.
6. The name and the icon differ from those of Qalculate!, and the store listing says that QCalc is not an official Qalculate! app.
7. libqalculate has no Japanese translation, so the engine's messages and list names are in English. The app translates the 28 most common messages and the category names, and adds Japanese names to functions, units and constants.

Limits of libqalculate itself are explained to users instead of being worked around: `limit()` always warns that its result may be wrong, `dsolve()` solves only first-order equations, infinite sums are not evaluated, there is no eigenvalue function, and one-letter names such as `a` or `m` may be read as units.

3. Architecture



Architecture · the page, Java, the engine process and storage

The user only touches the page in the WebView. Every calculation runs in libqalculate inside a separate engine process. The Java layer connects the two, supervises the engine, saves data and downloads exchange rates (basic design 2.1–2.3).

The parts meet only through three fixed interfaces, and each of them passes plain strings (JSON). This is what makes it possible to test the page, the Java code and the engine separately (section 9).

Interface	Direction	Contents
`QalcNative`	Page → Java	Eight methods that `JsApi` exposes to the WebView: `send`, `cancel`, `updateRates`, `loadState`, `saveState`, `copy`, `appInfo`, `openUrl`
`QalcBridge`	Java → page	Callbacks provided by `app.js`: `receiveText` for replies and `receiveEvent` for `ans` events (called with `evaluateJavascript`)

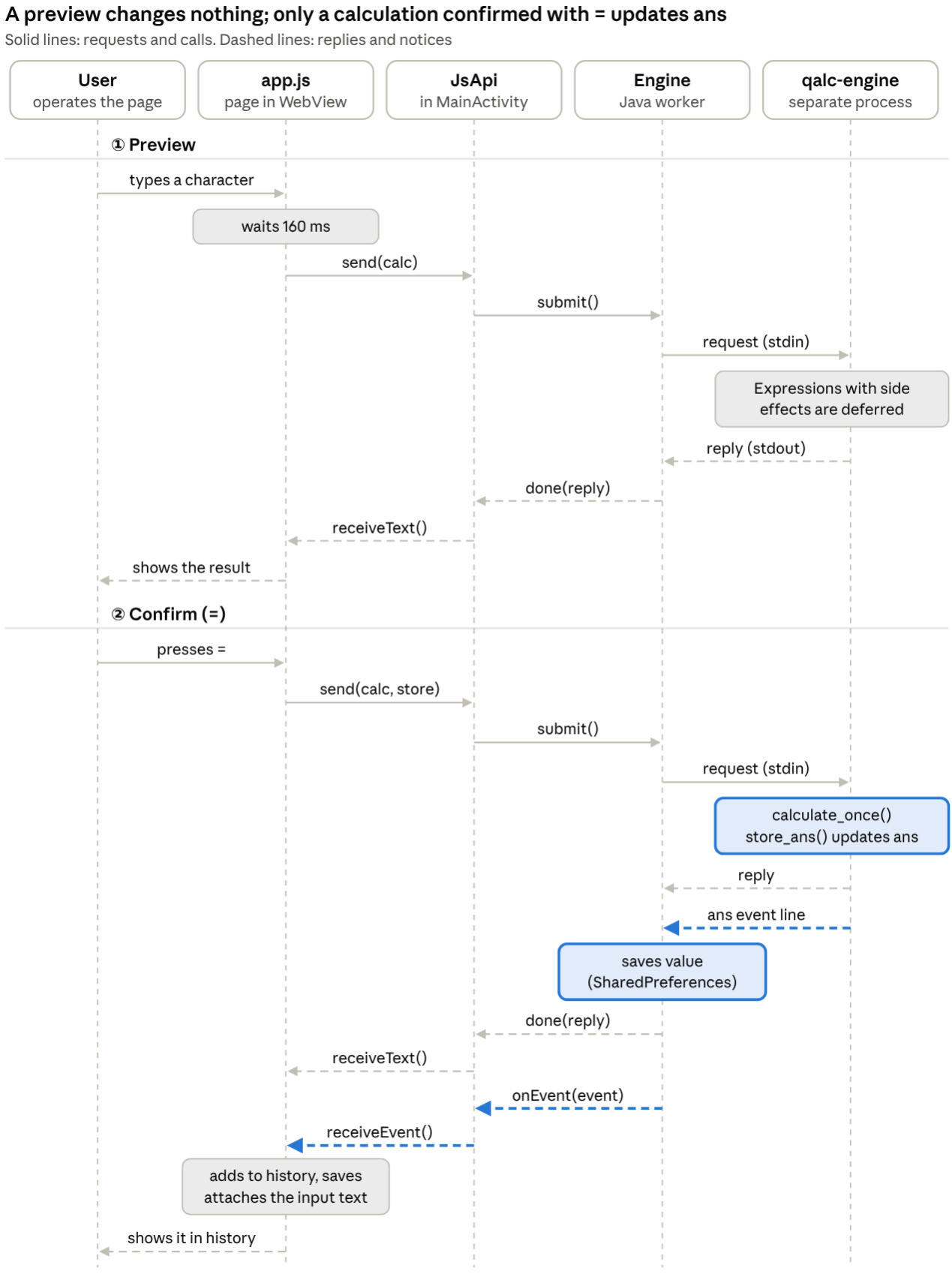
Interface	Direction	Contents
JSON lines	`Engine` ⇄ `qalc-engine`	One request per line on the engine's standard input; one reply or event per line on its standard output

File	Lines	Role
`app/assets/index.html`	51	Page layout: top bar, history, input field, keypad and menu containers
`app/assets/style.css`	704	Page look: light and dark colors, keys and tabs, matrix tables, text sizes, panels, graphs, landscape layout
`app/assets/app.js`	2,314	Page logic: keys and templates, input, preview, confirm, history, matrix tables, undo, examples, lists and search, 2D and 3D graphs, definitions, rates, settings, licenses
`MainActivity.java`	430	`MainActivity` and `JsApi`: the WebView and its settings, the JavaScript bridge, window insets (Android 15+), back handling, lifecycle
`Engine.java`	357	Starts, supervises and restarts the engine; matches replies to requests; saves and restores `ans`
`RatesUpdater.java`	164	Downloads, checks and replaces the exchange-rate files
`StateStore.java`	123	Writes the page state (history, settings, draft) to files at once
`bridge/qalc-engine.cc`	1,606	The engine's bridge: JSON protocol, calls into libqalculate, graph values, `ans`, watchdog, protection of definition files, Japanese units and `trace()`, closing message stops
libqalculate, GMP, MPFR, libxml2	—	The engine's core: official sources (5.13.2, 6.3.0, 4.2.2 and 2.15.4), linked statically

Deployment (basic design 2.2). Google Play receives the App Bundle signed with the upload key, builds an APK for each device (about 3.3 MB for arm64) and signs it with the app signing key that Google keeps. When the app is installed, Android extracts `libqalcengine.so` into the native library directory

(`android:extractNativeLibs="true"`). Despite its name, it is an executable: `Engine` starts it with `ProcessBuilder` as a child process in the working directory `files/`, with these environment variables: `HOME`; `QALCULATE_USER_DIR` (`files/qalculate/`, where libqalculate keeps user definitions and exchange rates); `TZ` (the current UTC offset in POSIX form, because a static Linux program cannot read Android's time-zone data); `LANG=C.UTF-8`; and `QALC_CURRENCY` (the currency of the phone's region, JPY in Japan).

4. Main flow: preview and confirm



Sequence diagram · live preview and confirmed calculation

The live preview and the confirmed calculation are both `calc` requests (detailed design 3.5). They differ only in four fields: `store` (whether `ans` is updated), `preview` (whether expressions with side effects are skipped), `timeout` and the maximum length of the result.

- **Preview**: 160 ms after the last key, the page sends `calc` with a 1.5 s limit. Expressions that would change something — assignments such as `x = 5`, `save()`, or user functions that save — are not run in a preview. The engine answers `deferred` instead, so typing never defines or saves anything by accident.
- **Confirm (=)**: `calc` with `store:true` and a 20 s limit. The engine evaluates the expression exactly once (`calculate_once`: the same steps as `calculateAndPrint` in `qalc`, but with a single evaluation), so random values or expressions like `a = a + 1` are never evaluated twice, and the value shown is the value stored.
- **ans**: after a confirmed calculation, the engine shifts `ans5 ← … ← ans` as `qalc` does and sends an `ans` event line after the reply. Java saves its `value` in SharedPreferences and gives it back with `set_ans` whenever a new engine process starts, so `ans` survives restarts. The event's `input` field is the exact text that the history inserts when the user taps a result (for example `7/3`, not the rounded `2.333333333`).

5. The engine protocol

The app writes one JSON request per line to the engine's standard input, and the engine answers with one JSON line on its standard output, with the same `id` (detailed design 3.3). Requests are handled one at a time.

- Success: `{"id":…, "ok":true, …}`. Failure: `{"id":…, "ok":false, "error":"…"}`.
- `"restart":true` in a reply means that the engine exits after the reply; Java starts a new process for the next request.
- Anything that libqalculate prints by itself is moved to standard error at start-up, so it never mixes with the replies.

Command	Main request fields	Main reply fields
`init`	—	Engine version, time of the exchange rates, whether `ans` is defined

Command	Main request fields	Main reply fields
`calc`	`expr`, `opts` (settings), `timeout` (100–120,000 ms), `store`, `preview`, `max_chars`	`result`, `parsed`, `approx`, `comparison`, `timeout`, `truncated`, `messages`, `deferred`
`set_ans`	`expr`	`set`
`plot`	`exprs` (up to 12), `min`, `max`, `steps`, `var`, `timeout`	`series` (x and y values for each expression), `messages`
`plot3d`	`exprs` (up to 4), `xmin`, `xmax`, `ymin`, `ymax`, `nx` and `ny` (2–121), `timeout`	`x`, `y`, `series` (z values on the grid, `complete`), `messages`
`catalog`	`kind` (functions, variables or units)	`items`: names, titles, categories, descriptions and the text to insert
`rates_info`, `rates_reload`	—	Sources and file names; whether new rates were loaded
`define_var`, `define_func`, `delete_def`, `user_defs`	A name and a value or an expression	`saved`; the list of user definitions

```
{"id":19,"cmd":"calc","expr":"7/3","opts":
{"approx":"auto","precision":10},"timeout":20000,"store":true,"max_chars":450
000}
{"id":19,"ok":true,"result":"2 + 1/3 ≈ 2.333333333","parsed":"7 /
3","approx":false,"comparison":false,"timeout":false,"truncated":false,"messa
ges":[]}
{"event":"ans","id":19,"value":"7/3","input":"7/3"}
```

A workaround for libqalculate 5.13.2 (detailed design 3.19). After a definite integral with an infinite bound, libqalculate leaves one level of “message stop” (`beginTemporaryStopMessages`) open, and from then on no warning or error is shown; the official `qalc` behaves the same way. The bridge program closes any open message stop before every command, so QCalc keeps showing warnings such as division by zero. libqalculate itself is not changed.

6. Time limits and recovery

Time limits work in three layers (detailed design 3.13). The innermost one normally ends a request; the outer ones are safety nets.

Request	Engine limit (`timeout`)	Engine watchdog (+3 s)	Java wait (timeout × expressions + 10 s)	Page's last safety net
Preview	1.5 s	4.5 s	11.5 s	120 s
Confirm (=)	20 s	23 s	30 s	120 s
2D graph (6 expressions)	12 s, shared	15 s	82 s	204 s
3D graph (4 expressions)	15 s, shared	18 s	70 s	180 s
Lists, definitions, `init`	—	—	30 s	120 s

- The **watchdog** thread inside the engine checks every 0.2 s. When a deadline has passed, it writes a reply with `timeout:true` and `restart:true` and exits. This is faster than waiting for libqalculate's own abort.
- **Java** (`Engine`) runs one engine process and handles one request at a time. If no reply arrives in time, it kills the process and starts a new one for the next request.

Situation	What Java does	What the page gets
The reply has `restart:true`	Passes the reply on, then stops the process	The engine's reply
No reply in time	Kills the process	Code `timeout`
The user cancels (taps "calculating…")	Kills the process	Code `cancelled`
The engine exits while working on a request	Never resends that request, because it would probably crash again	Code `crash`
Writing the request fails because the engine had already exited	Starts a new engine and sends the request once more	The normal reply, or code `engine`

After a restart, Java restores `ans` with `set_ans`, and libqalculate reloads the user definitions from its files. The user notices nothing, except that `ans2` to `ans5` and temporary variables (such as `x = 5`) are gone.

Since 1.2.0, `set_ans` reads the value with as many digits of precision as it has: a new engine starts at precision 10, and before 1.2.0 a restored approximate `ans` kept only about 38 digits. Numbers of 2,000 digits or more are read directly with GMP and passed in as temporary variables, because libqalculate reads digits one at a time, in time that grows with the square of their number (200,000 digits: 0.2 s instead of 3.4 s on a PC, with exactly the same result).

7. Data, external interfaces and security

All data stays in the app's private storage (basic design 2.7). Every file is first written to a temporary file and then renamed, so a forced stop leaves either the old or the new content.

Data	Where	Notes
History	`state/qalc.history.json`	Up to 300 items. The newest results are kept whole up to 1,000,000 characters in total; older long results keep their first 2,000 characters
Settings	`state/qalc.settings.json`	17 items
Expression being typed	`state/qalc.draft.json`	Saved 0.5 s after typing
Graph inputs	`state/qalc.plot.*.json`, `state/qalc.plot3d.*.json`	Expressions, ranges, grid
`ans`	SharedPreferences `engine`, key `ans`	Up to 500,000 characters (an approximate value is saved as interval(lower, upper): about 400,000 characters for 200,000 digits)
User-defined variables and functions	`qalculate/definitions/*.xml`	libqalculate's XML format. Unreadable files are moved to `qalculate/broken-definitions/`

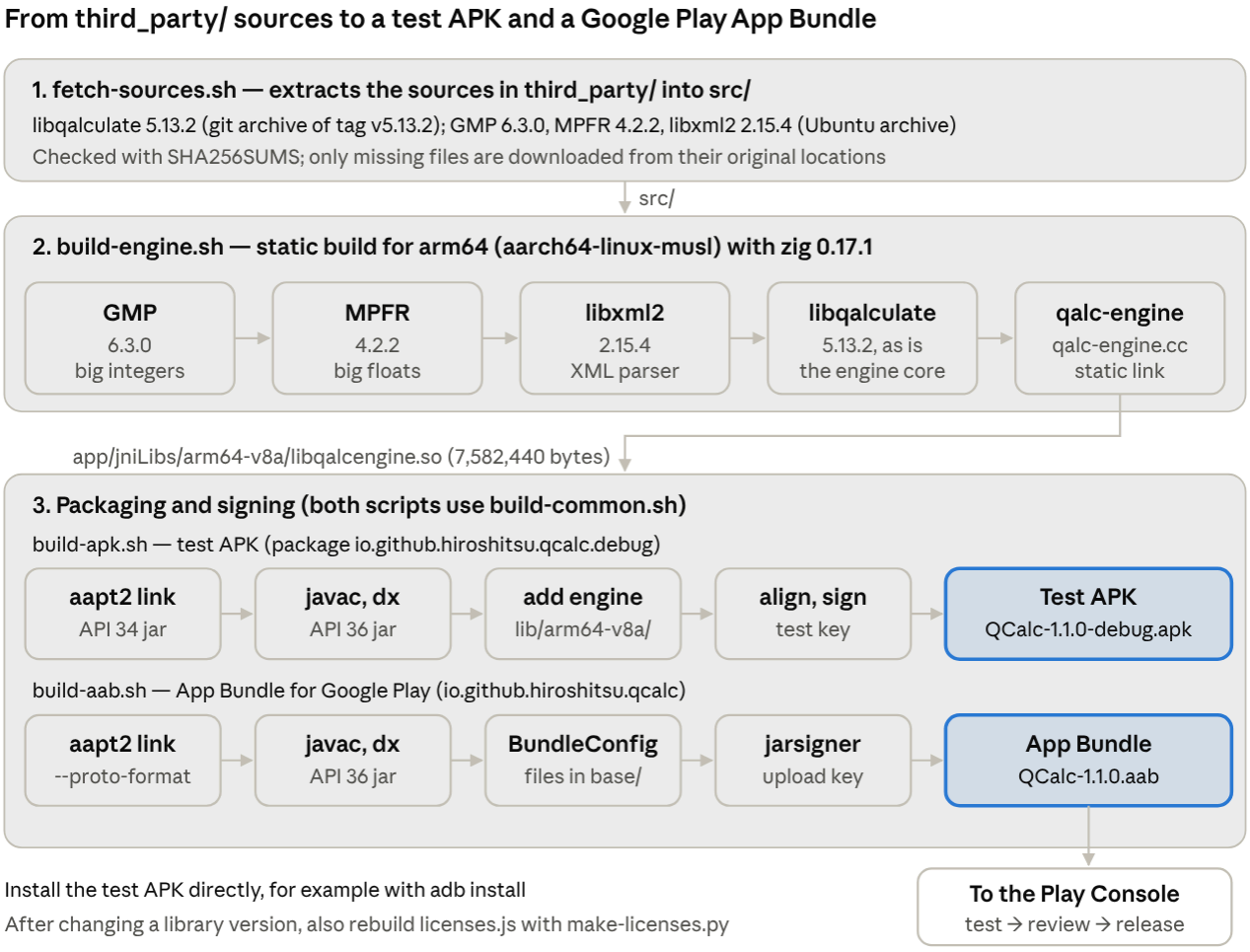
Data	Where	Notes
Exchange rates	`qalculate/eurofxref-daily.xml`, `btc.json`, `rates.json`	As downloaded

The app's only network traffic is downloading exchange rates over HTTPS from the same three sources that Qalculate! uses (basic design 2.8): the European Central Bank (major currencies), Coinbase (Bitcoin), and currency-api through jsDelivr (other currencies, with a fallback host). The User-Agent is `QCalc/1.2.0 (Android; libqalculate 5.13.2)`. Connections time out after 15 s and reads after 20 s, and replies over 1,000,000 bytes are discarded.

Security and privacy (basic design 2.10):

- The WebView shows only the bundled pages under `file:///android_asset/`. File and content access are off, and other links open in the browser only if they are http or https.
- Results and user input are inserted with `textContent`, never as HTML.
- The JavaScript bridge checks every request as JSON and rejects requests that contain line breaks. Saved-state names are limited to 64 lowercase letters, digits and `._-`.
- libqalculate is built with `--disable-insecure`, which disables `command()`. The engine reads and writes only `files/qalculate/`.
- The app has only two permissions (INTERNET and VIBRATE), collects no personal data, and has no ads or analytics. The privacy policy is at https://hiroshitsu.github.io/qcalc/privacy.html.
- The upload key and its password are never published or committed. Google keeps the app signing key.

8. Build and release



Build flow · three stages to the test APK and the App Bundle

Three shell scripts build everything on Ubuntu 24.04 (build and maintenance manual 5.2–5.6). Neither the Android NDK nor Gradle is needed. An Android Studio (Gradle) setup for the same sources is also included, but it has not been tested yet.

- **Engine**: zig 0.17.1 cross-compiles GMP, MPFR, libxml2 and libqalculate as static libraries (-O2) for `aarch64-linux-musl`, then links `qalc-engine.cc` (C++17) statically with `-Wl,-z,max-page-size=16384`, so that the engine also runs on devices with 16 KB memory pages. The result is a 7,587,080-byte executable.
- **libqalculate configure options**: `--enable-compiled-definitions` (definitions built into the library), `--disable-insecure` (no `command()`), `--without-libcurl` (Java downloads the rates), `--without-icu`, `--without-gnuplot-call` (the page draws the graphs), `--with-readline=no` and `--disable-nls`.
- **Outputs**: `QCalc-1.2.0-debug.apk` (3,299,412 bytes; a separate package, `io.github.hiroshitsu.qcalc.debug`, signed with a test key) and `QCalc-1.2.0.aab` (3,285,758 bytes, signed with the upload key, SHA-256 6134f25ed3d3fa25dc275f2e8f66b69717583f29536ea883e941aa519dda9bf6). The

bundle is validated with bundletool 1.17.2. Its BundleConfig turns off “uncompressed native libraries”, so that the APKs that Google Play makes still extract the engine as a file.

- **Updating libqalculate** (build and maintenance manual 5.9): put the new tagged source in `third_party/`, rebuild, port any upstream changes to the steps copied from `qalc` (`calculate_once`, `store_ans`), check the exchange-rate sources, regenerate the license texts, and rerun all the tests, including the comparison with `qalc`.
- **Releasing an update**: raise `VERSION_CODE` in `version.properties`, build the bundle, test the debug APK on a phone, and upload the bundle to a Play Console track.

9. Testing

No physical device was available, so each of the four parts was tested on a PC running Ubuntu 24.04 (test specification 6.1):

- **Engine**: the same arm64 engine binary that ships in the APK, run with `qemu-aarch64-static` and compared with the official `qalc` built from the same libqalculate tag.
- **Java**: `Engine.java` and `StateStore.java` run unchanged on OpenJDK 21 with small stand-ins for Android's `Context` and `SharedPreferences`, against both the real engine and a fake one.
- **Page**: the real `app/assets` in Chromium (Playwright) at the size of a phone screen, with requests relayed to the real engine.
- **Packages**: apksigner, zipalign, aapt2, jarsigner and bundletool on the APK and the App Bundle.

Area	Items	Passed
Comparison with the official `qalc`	309	309
Engine functions, 3D graph values, calculus and matrices, Japanese units	66	66
All built-in examples	73	73
Java (real engine and fake engine)	24	24
Page: keypads, calculus and matrices, examples, undo, text size, search, history, 3D graphs, fixes	89	89
APK and App Bundle checks	22	22
High precision (1.2.0): 200,000 digits compared digit by digit with mpmath, limits, restoring a long ans	16	16

Area	Items	Passed
Java (1.2.0): ans kept with all its digits	7	7
Page (1.2.0): long results shown, copied and saved	32	32
Total	**638**	**638**

In addition, a build of the engine with AddressSanitizer and UndefinedBehaviorSanitizer ran the functional tests and 33,000 random requests, and the release engine ran 6,000 random requests, with no errors. For 1.2.0, the comparison with qalc, the Java tests, 21 page tests and the 22 package checks were repeated with the new build, and the 55 new items in the table were added.

Not tested yet (test specification 6.6): installing and running on a real device, TalkBack, a real exchange-rate download, input with a real Japanese keyboard, rendering in the Android System WebView, window insets and predictive back on a device with Android 15 or later, the Android Studio build, a device with 16 KB pages, and Google Play's review and closed test.

10. Guide to the Japanese documents

Document	What it covers
Overview (概要), `00-overview.pdf`	Changes in 1.2.0 and 1.1.0, list of the documents, the big picture, components and versions, glossary, revision history
1. Requirements definition (要件定義書), `01-requirements.pdf`	Purpose, scope, 19 use cases, 29 functional requirements, non-functional requirements, constraints, 19 acceptance criteria
2. Basic design (基本設計書), `02-basic-design.pdf`	Components, deployment, modules, screens and transitions, keypads, main flows, data, external interfaces, error handling, security
3. Detailed design (詳細設計書), `03-detailed-design.pdf`	Class diagram, JavaScript bridge, engine protocol, calculation, engine supervision, page modules, input, history, graphs, exchange rates, data formats, time limits, error codes, Android 13–16, licenses, App Bundle, search, additions to the engine

Document	What it covers
4. User manual (操作マニュアル), `04-user-manual.pdf`	Installation, the screen, calculating, calculus, matrices, keyboard input, examples, menu, settings, troubleshooting, with screenshots
5. Build and maintenance manual (ビルド・保守マニュアル), `05-build-and-maintenance.pdf`	Environment, build flow, sources, engine build, APK, App Bundle, releases, checks, updating libqalculate, maintenance notes, Android Studio
6. Test specification and results (テスト仕様書・結果), `06-test-specification.pdf`	Test policy and environment, engine tests, Java tests, page tests, package checks, tests not yet done, known limits

11. Glossary

Term	Meaning
libqalculate	The calculation library of the official Qalculate!. It does every calculation in QCalc
Engine	The bridge program `qalc-engine` linked with libqalculate. It ships as `lib/arm64-v8a/libqalcengine.so` and runs as a child process
Preview	The result shown while an expression is being typed
Confirm	Pressing “=”: the expression is calculated with `store:true` and the result is added to the history
`ans` event	The line that the engine sends after a confirmed calculation, with the new `ans` (`value`) and the text to insert (`input`)
Watchdog	The thread inside the engine that answers with a timeout and exits when a calculation runs past its limit + 3 s
Message stop	libqalculate's way of holding back warnings and errors during a calculation (`beginTemporaryStopMessages`)

Term	Meaning
Template	A function call with its arguments laid out, inserted by a key (for example `integrate(, 0, 1)`), with the cursor where the expression goes
Upload key	The key that signs the App Bundle uploaded to Google Play. Google keeps the key that signs the app itself (Play App Signing)